

Linguaggi Formali e Compilatori

(Formal Languages and Compilers)

prof. S. Crespi Reghizzi, prof. Angelo Morzenti
(prof. Luca Breveglieri)

Prova scritta - 6 settembre 2013 - Parte I: Teoria

CON SOLUZIONI - A SCOPO DIDATTICO LE SOLUZIONI SONO MOLTO ESTESE E COMMENTATE VARIAMENTE - NON SI RICHIEDE CHE IL CANDIDATO SVOLGA IL COMPITO IN MODO ALTRETTANTO AMPIO, BENSÌ CHE RISPONDA IN MODO APPROPRIATO E A SUO GIUDIZIO RAGIONEVOLE

AVVISO - GLI ESERCIZI DI ANALISI SINTATTICA VANNO AFFRONTATI SECONDO LA TEORIA

ELR / ELL - SI VEDA IL SITO WEB DEL CORSO PER MATERIALE DIDATTICO E INFORMAZIONI

NOME:

MATRICOLA:

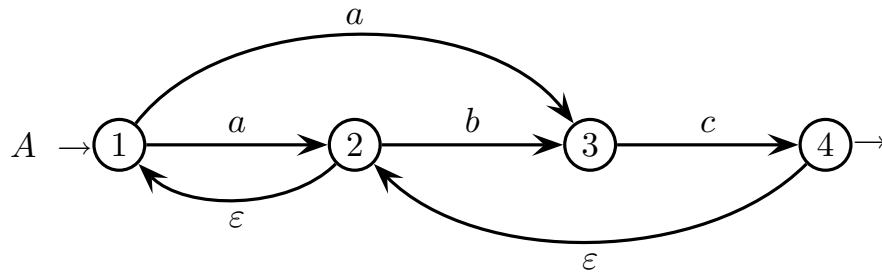
FIRMA:

ISTRUZIONI - LEGGERE CON ATTENZIONE:

- L'esame si compone di due parti:
 - I (80%) Teoria:
 1. espressioni regolari e automi finiti
 2. grammatiche libere e automi a pila
 3. analisi sintattica e parsificatori
 4. traduzione sintattica e analisi semantica
 - II (20%) Esercitazioni Flex e Bison
- Per superare l'esame l'allievo deve sostenere con successo entrambe le parti (I e II), in un solo appello oppure in appelli diversi, ma entro un anno.
- Per superare la parte I (teoria) occorre dimostrare di possedere conoscenza sufficiente di tutte le quattro sezioni (1-4), rispondendo alle domande obbligatorie.
- È permesso consultare libri e appunti personali.
- Per scrivere si utilizzi lo spazio libero e se occorre anche il tergo del foglio; è vietato allegare nuovi fogli o sostituirne di esistenti.
- Tempo: Parte I (teoria): 2h.15m - Parte II (esercitazioni): 60m

1 Espressioni regolari e automi finiti 20%

1. Si consideri l'automa finito A seguente (nondeterministico) con alfabeto $\{a, b, c\}$:



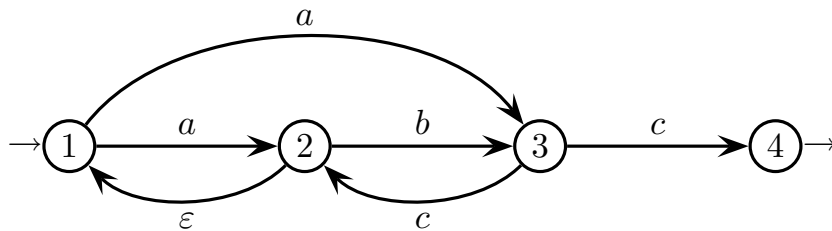
Si risponda alle domande seguenti:

- (a) Si trovi un automa finito B equivalente ad A , ma senza transizioni spontanee.
- (b) Si trovi un automa finito deterministico C equivalente ad A .
- (c) Si trovi un automa finito deterministico D che riconosce il linguaggio complemento $\overline{L(A)}$, si verifichi se esso è minimo e, se necessario, lo si minimizzi.
- (d) (facoltativa) Si ricavi un'espressione regolare R non ambigua che genera il linguaggio $L(A)$.

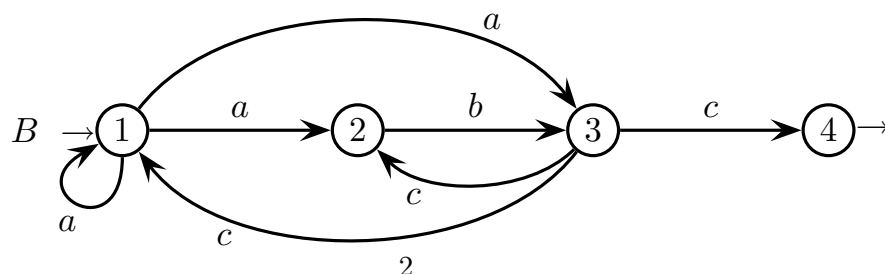
Soluzione

- (a) Automaton A is clean. However it is nondeterministic because of these features: there are two spontaneous transitions and state 1 has multiple (double) arcs with the same label a . Now we cut the spontaneous transitions.

Cut transition $4 \xrightarrow{\varepsilon} 2$ and duplicate the transitions that go into state 4:

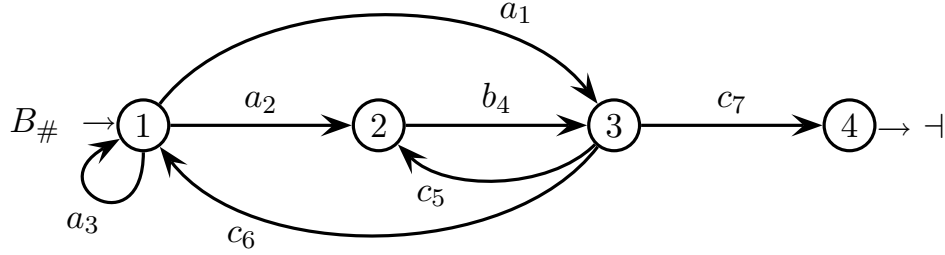


Cut transition $2 \xrightarrow{\varepsilon} 1$ and duplicate the transitions that go into state 2:

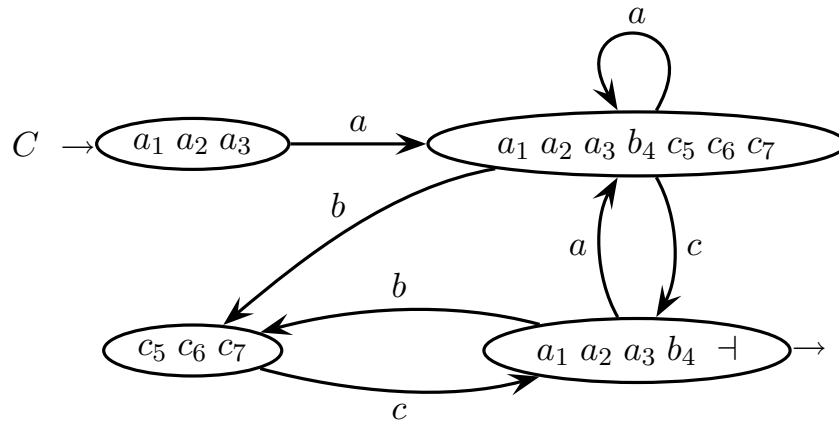


Automaton B is equivalent to A but without spontaneous transitions, and is not deterministic yet.

- (b) Automaton B does not have any spontaneous transitions. However it has more multiple (triple) arcs on state 1 and new multiple arcs (triple) on state 3, with respect to A . To determinize, we can use the Berry-Sethi algorithm and directly apply it to automaton B :

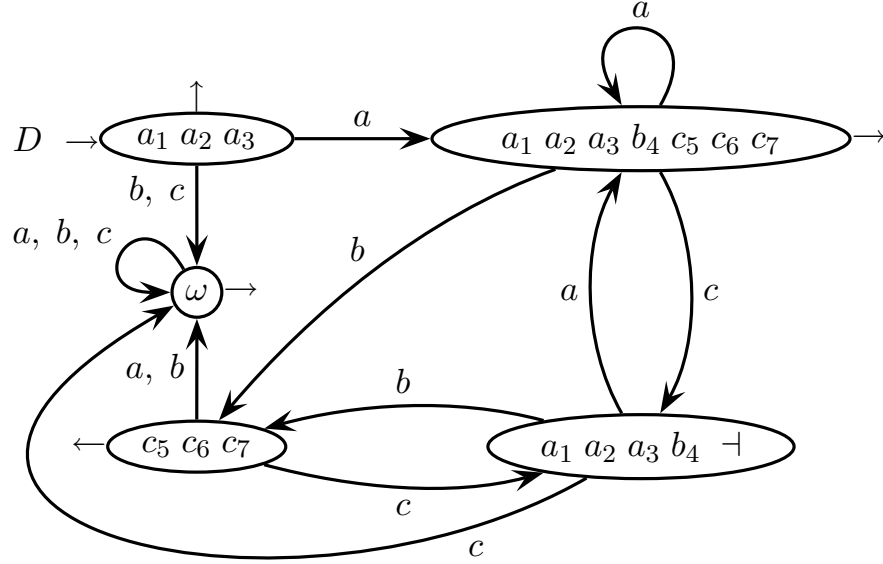


<i>initials</i>	a_1 a_2 a_3
<i>generator</i>	<i>followers</i>
a_1	c_5 c_6 c_7
a_2	b_4
a_3	a_1 a_2 a_3
b_4	c_5 c_6 c_7
c_5	b_4
c_6	a_1 a_2 a_3
c_7	$\dagger$



Automaton C is equivalent to A and is fully deterministic.

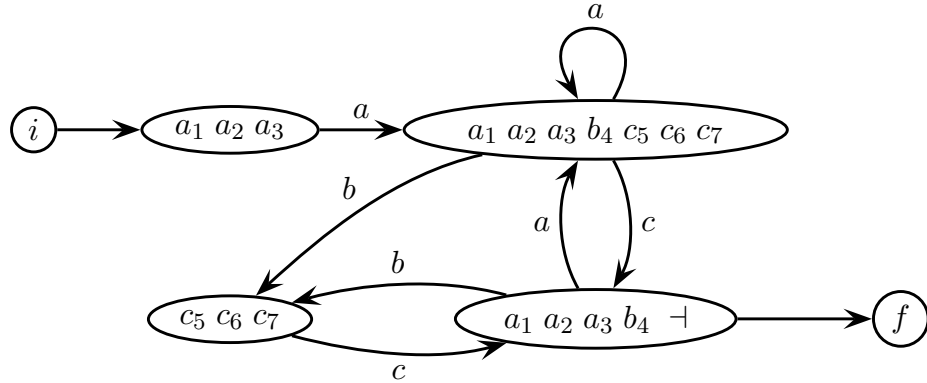
- (c) The complement deterministic automaton D is quickly obtained by completing the deterministic automaton C , that is by introducing the error state ω , and then by switching the final and non-final states of the completed automaton (still deterministic):



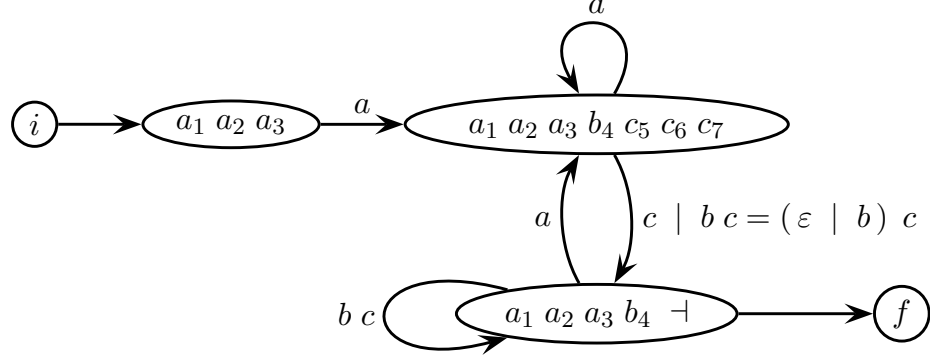
Automaton D is deterministic and minimal: state ω is distinguishable from any other state as it comes from making final the error state; the non-final state is distinguishable from the three final states other than state ω ; and these three final states are all distinguishable from one another because with some arc labels they pairwise go to distinguishable states.

To make a more formal verification, we can draw the state table and check the equivalence condition for each state pair (left to the reader).

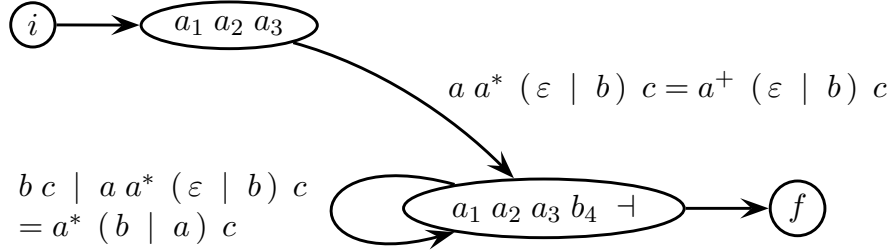
- (d) A way to obtain a non-ambiguous regular expression R such that $L(R) = L(A)$, is to eliminate the nodes of the deterministic automaton C (Brzozowski).
Make the initial and final states unique, and respectively without ingoing and outgoing arcs:



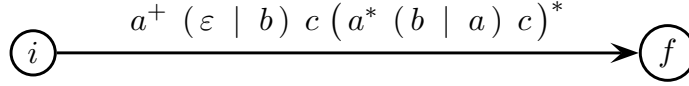
Eliminate node $c_5 c_6 c_7$:



Eliminate node $a_1 a_2 a_3 b_4 c_5 c_6 c_7$:



Eliminate nodes $a_1 a_2 a_3 b_4 -$ and $a_1 a_2 a_3$:



Therefore a regular expression R is the following:

$$R = a^+ (\varepsilon \mid b) c (a^* (b \mid a) c)^*$$

Expression R is non-ambiguous by construction, as it is derived by a deterministic automaton through a node elimination procedure that maps paths onto (sub)expressions.

A final observation: in question (a) the technique chosen for eliminating the spontaneous transitions deserves some care. For instance if we wanted to cut transition $4 \xrightarrow{\varepsilon} 2$ and to duplicate the transitions that go out from state 2, then we should duplicate also the spontaneous transition $2 \xrightarrow{\varepsilon} 1$. So there will be two spontaneous transitions left, one old and one new. Going on and cutting more, such transitions have to decrease and eventually all will disappear, but this would require more cut steps. In the end any correct cut works, but the efficiency depends on the order.

2 Grammatiche libere e automi a pila 20%

1. Il linguaggio L delle espressioni polacche prefisse con i due operatori $\oplus$ e $\ominus$, entrambi binari (ossia con due operandi), e con l'operando a , è definito dalla grammatica G seguente (assioma S):

$$G: S \rightarrow \oplus S S \mid \ominus S S \mid a$$

Sono inoltre date le due espressioni regolari R_1 e R_2 seguenti:

$$R_1 = (\oplus \mid a)^* \ominus (\oplus \mid a)^*$$

$$R_2 = (\oplus a^* \ominus a^*)^*$$

Si risponda alle domande seguenti:

- (a) Si spieghi a parole qual è la condizione espressa dall'espressione regolare R_1 . Si scriva una grammatica non estesa e non ambigua G_1 per il linguaggio intersezione $L \cap L(R_1)$, e si disegni l'albero sintattico della stringa " $\oplus \oplus a \ominus a \oplus a a a$ ".
- (b) (facoltativa) Si scriva una grammatica non estesa G_2 per il linguaggio intersezione $L \cap L(R_2)$.

Soluzione

- (a) The strings of the regular language $L(R_1)$ contain exactly one operator $\ominus$, and as for the rest they are completely free. Thus every sentence of the intersection language $L \cap L(R_1)$ is a phrase of L that contains exactly one operator $\ominus$. This filters out expression a , which belongs to L . The shortest sentence is " $\ominus a a$ ".

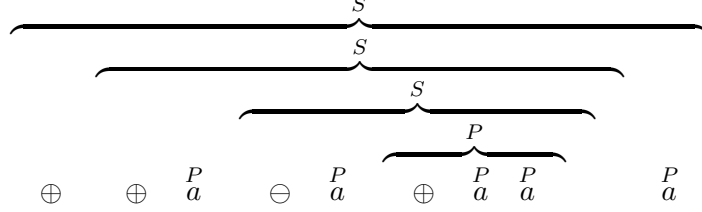
To obtain grammar G_1 , we refine the nonterminal S and we introduce a second nonterminal P , which generates the expressions that contain only the operator $\oplus$. The form of the rules is basically the same as in the grammar G since we still want to generate expressions in the prefix form, but we have to differentiate the use of S and P . So we have a grammar G_1 as below (axiom S):

$$G_1 \begin{cases} S \rightarrow \oplus P S \mid \oplus S P \mid \ominus P P \\ P \rightarrow \oplus P P \mid a \end{cases}$$

A quick look at the rules of grammar G_1 should convince us that the grammar is correct. As the rules have the same form as those of G , they generate expressions of L ; but every sentential form contains exactly one S (and possibly many P 's); so when this one S generates an operator $\ominus$, which fact sooner or later must happen for the derivation to terminate, then there will remain no more S 's and the derivation will generate only operators $\oplus$ until it terminates. Thus the final

terminal string is a phrase of L with exactly one operator $\ominus$. Furthermore all such restricted prefix expressions are generated, since grammars G and G_1 basically have the same rule forms (the well known polish prefix grammar).

The syntax tree of the valid sample string “ $\oplus \oplus a \ominus a \oplus a a a$ ” is the following, represented as a parenthesized string:



Such a grammar G_1 is not ambiguous, as it is easy to see that it is deterministic $LR(1)$. Imagine to perform a bottom-up syntactic analysis; to do so, locate the position of the leftmost handle (right part of a rule) in the string, then reduce the handle to the left part of the rule and proceed likewise until the string reduces to the axiom or until an error occurs. It is clear that this procedure is deterministic: decomposing a prefix expression is uniquely determined; and the alternatives $\oplus P S$ and $\oplus S P$ can be distinguished from their allowing the operator $\ominus$ to occur either in the second or in the first operand.

- (b) In the strings of the regular language $L(R_2)$, the signs $\oplus$ and $\ominus$ alternate, and the leftmost and rightmost signs are $\oplus$ and $\ominus$, respectively; as for the rest, the strings of $L(R_2)$ are completely free. So every sentence of the intersection language $L \cap L(R_2)$ is a phrase of L that has alternating operators, with a leftmost operator $\oplus$ and a rightmost operator $\ominus$. This implies that the sentence contains two or more operators, always in even number (two, four, six, etc). The shortest sentences are “ $\oplus a \ominus a a$ ” and “ $\oplus \ominus a a a$ ”.

To obtain grammar G_2 , we introduce a nonterminal S_{PM} (short for $S_{Plus\ Minus}$) that generates the expressions that alternate the operators and have leftmost and rightmost operators $\oplus$ and $\ominus$, respectively. We likewise introduce three more nonterminals S_{MP} , S_{PP} and S_{MM} that work in a similar way; their subscripts tell how. Notice that the nonterminals S_{PP} and S_{MM} necessarily generate expressions with an odd number of operators (one, three, five, etc), as they must also have alternating operators. Thus grammar G_2 is the following (axiom S):

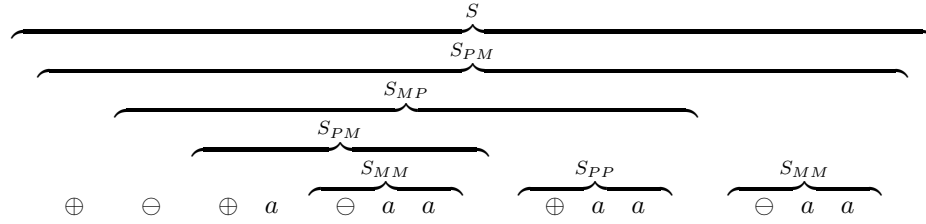
$$G_2 \left\{ \begin{array}{l} S \rightarrow S_{PM} \\ S_{PM} \rightarrow \oplus S_{MM} S_{PM} \mid \oplus S_{MP} S_{MM} \mid \oplus a S_{MM} \mid \oplus S_{MM} a \\ S_{MP} \rightarrow \ominus S_{PM} S_{PP} \mid \ominus S_{PP} S_{MP} \mid \ominus a S_{PP} \mid \ominus S_{PP} a \\ S_{PP} \rightarrow \oplus S_{MM} S_{PP} \mid \oplus S_{MP} S_{MP} \mid \oplus a S_{MP} \mid \oplus S_{MP} a \mid \oplus a a \\ S_{MM} \rightarrow \ominus S_{PM} S_{PM} \mid \ominus S_{PP} S_{MM} \mid \ominus a S_{PM} \mid \ominus S_{PM} a \mid \ominus a a \end{array} \right.$$

Grammar G_2 is correct. For instance consider rule $S_{PM} \rightarrow \oplus S_{MM} S_{PM}$: it expands the left nonterminal S_{PM} , i.e., it is expected to generate a subexpression with alternating operators and with leftmost-rightmost plus-minus operators; coherently with this requirement, its right part has a leftmost terminal $\oplus$ and a rightmost nonterminal S_{PM} that is expected to generate a subexpression with a rightmost operator $\ominus$; at the concatenation of nonterminals S_{MM} and S_{PM} , we

see that the expected adjacent operators alternate; and likewise at the concatenation of the leading terminal $\oplus$ and the adjacent nonterminal S_{MM} . The clear symmetry of the rules helps us see that all the cases have been included.

This analysis can be carried out for all the other alternative rules that expand nonterminal S_{PM} , which cover all the possible cases including the terminal ones. We can similarly analyze also all the alternative rules that expand the other three nonterminals S_{MP} , S_{PP} and S_{MM} . Notice that symbols S_{PM} and S_{MP} generate sentences with two, four, six, $\dots$, operators; while symbols S_{PP} and S_{MM} generate sentences with one (when the leftmost and rightmost operators coincide), three, five, $\dots$, operators. Since the axiom S must expand to a symbol S_{PM} , the language $L(G_1)$ is the required one.

To illustrate we show the syntax tree (in a parenthesized form) of the valid string “ $\oplus \ominus \oplus a \ominus a a \oplus a a \ominus a a$ ”, as follows:



Grammar G_2 is not ambiguous, essentially for the same reasons as those of grammar G_1 . In fact the two grammars basically have the same rule forms, though grammar G_2 has more nonterminals and combinations. However distinguishing the right parts of the rules is deterministic also in G_2 , since the leftmost and rightmost operators always give a unique identification.

2. Un ipotetico linguaggio di descrizione di grafica vettoriale (p.es. Picture di MS Word) comprende le seguenti figure di base: triangolo (*triang*) e poligono (*polygon*).

Tutte le figure base devono iniziare con un attributo *points* per elencare, tra virgolette e separate da barra, coppie di numeri naturali (separati da virgola) che rappresentano le coordinate dei vertici (3 coppie per il triangolo e > 3 coppie per il poligono).

Inoltre ogni figura base può avere un attributo *style* per elencarne gli aspetti grafici seguenti, ciascuno nella forma "*aspetto:valore*" e separati da punto-e-vigola:

fill colore di riempimento; i valori di colore possibili sono *black*, *red*, *green* e *none*; si supponga che a ogni colore corrisponda un simbolo terminale opportuno

stroke colore della linea di contorno (con gli stessi valori di cui sopra)

width spessore della linea di contorno, il cui valore è un numero naturale; si supponga che il simbolo terminale *n* rappresenti un numero naturale qualunque

Ciascun aspetto è facoltativo, ma la lista di aspetti - se l'attributo *style* è presente - non è vuota; l'ordine di elencazione degli aspetti è fissato come sopra.

Esempi di figure base:

```
triangle points = "1,2 / 3,4 / 5,6" end triangle
```

```
triangle points = "1,3 / 5,7 / 4,6"  
  style = fill:red; stroke:red; width:3  
end triangle
```

```
polygon points = "1,1 / 2,3 / 2,5 / 6,5"  
  style = fill:green; width:2  
end polygon
```

Tra le figure possibili ci sono anche i poligoni a due livelli, nei cui vertici si trovano figure base. Le descrizioni di poligoni a due livelli devono contenere un numero ≥ 3 di figure base, descritte però solo nella loro geometria ossia senza attributo *style*, e possono contenere un solo attributo *style* valido per tutta la figura a due livelli. Esempio:

```
poly2level  
  
  triangle points = "1,3 / 5,7 / 4,6" end triangle  
  
  polygon points = "2,3 / 5,9 / 4,7 / 8,5" end polygon  
  
  polygon points = "1,3 / 5,7 / 4,6 / 15,16 / 2,7" end polygon  
  ...  
  style = fill:green; stroke:red; width:2  
  
end poly2level
```

Infine tra le figure possibili ci sono i gruppi, che raccolgono un numero ≥ 2 di figure di qualsiasi tipo: base, poligoni a due livelli e gruppi. Esempio:

```
group

    polygon ... end polygon

    group ... end group

    poly2level ... end poly2level

end group
```

In conclusione una stringa del linguaggio consiste in una sequenza non vuota di figure di qualunque tipo (base, poligoni a due livelli e gruppi).

Si risponda alle domande seguenti:

- (a) Si scriva una grammatica G_1 estesa (*EBNF*) e non ambigua che modelli il linguaggio tratteggiato sopra, ma supponendo che la sola figura base sia il triangolo e che non ci siano poligoni a due livelli.
 - (b) (facoltativa) Si scriva una grammatica G_2 (come prima estesa (*EBNF*) e non ambigua) comprendente anche il poligono base e il poligono a due livelli.
-

Soluzione

(a) Here is grammar G_1 (axiom `FIG_LST`):

$\langle \text{FIG_LST} \rangle \rightarrow \langle \text{FIGURE} \rangle^+$
$\langle \text{FIGURE} \rangle \rightarrow \langle \text{GROUP} \rangle \mid \langle \text{TRIANGLE} \rangle$
$\langle \text{GROUP} \rangle \rightarrow \text{group } \langle \text{FIGURE} \rangle^2 \langle \text{FIGURE} \rangle^* \text{ end group}$
$\langle \text{TRIANGLE} \rangle \rightarrow \text{triangle } \langle \text{POINT3} \rangle [\langle \text{STYLE} \rangle] \text{ end triangle}$
$\langle \text{POINT3} \rangle \rightarrow \text{points} = "n,n / n,n / n,n"$
$\langle \text{STYLE} \rangle \rightarrow \text{style} = \langle \text{ASPECT_LST} \rangle$
$\langle \text{ASPECT_LST} \rangle \rightarrow$ $\quad \langle \text{FILL} \rangle ; \langle \text{STROKE} \rangle ; \langle \text{WIDTH} \rangle$ $\quad \mid \langle \text{FILL} \rangle ; \langle \text{STROKE} \rangle$ $\quad \mid \langle \text{FILL} \rangle ; \langle \text{WIDTH} \rangle$ $\quad \mid \langle \text{STROKE} \rangle ; \langle \text{WIDTH} \rangle$ $\quad \mid \langle \text{FILL} \rangle \mid \langle \text{STROKE} \rangle \mid \langle \text{WIDTH} \rangle$
$\langle \text{FILL} \rangle \rightarrow \text{fill} : \langle \text{COLOR} \rangle$
$\langle \text{STROKE} \rangle \rightarrow \text{stroke} : \langle \text{COLOR} \rangle$
$\langle \text{WIDTH} \rangle \rightarrow \text{width} : n$
$\langle \text{COLOR} \rangle \rightarrow \text{black} \mid \text{red} \mid \text{green} \mid \text{none}$

As usual the square brackets mean optionality and power means repetition for an exact number of times. Grammar G_1 is extended and not ambiguous, as it consists of non-ambiguous components.

Notice that the rule `ASPECT_LST` is non-ambiguous and that it could be shortened by factoring in this way (among the others):

$$\begin{aligned}
 \langle \text{ASPECT_LST} \rangle \rightarrow & \langle \text{FILL} \rangle [; \langle \text{STROKE} \rangle] [; \langle \text{WIDTH} \rangle] \\
 & \mid \langle \text{STROKE} \rangle [; \langle \text{WIDTH} \rangle] \\
 & \mid \langle \text{WIDTH} \rangle
 \end{aligned}$$

This rule form still generates all the possible ordered combinations, except the empty string, in a non-ambiguous way, though it is somewhat less intuitive.

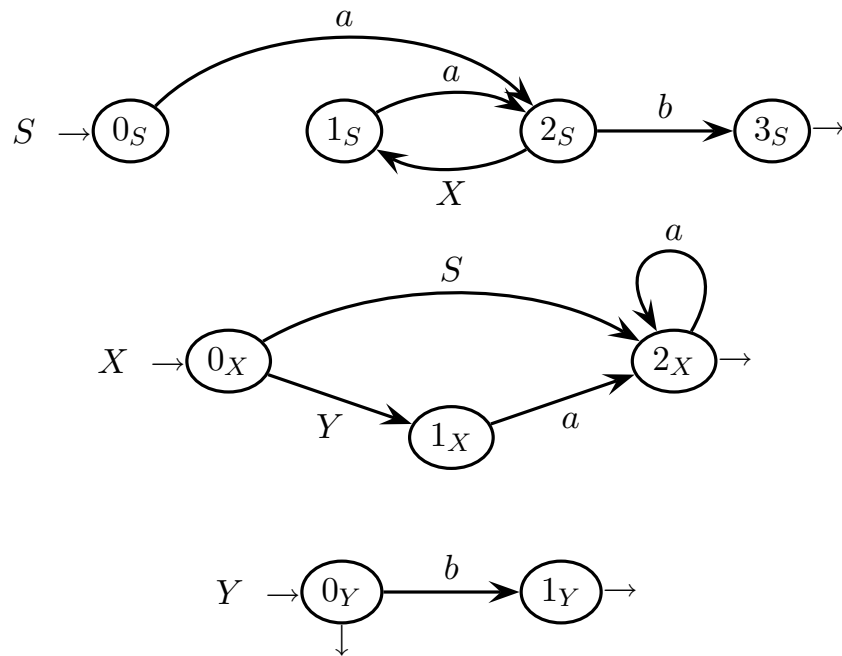
- (b) Here is grammar G_2 (axiom FIG_LST), derived from grammar G_1 with a few added parts in red:

$\langle \text{FIG_LST} \rangle$	$\rightarrow$	$\langle \text{FIGURE} \rangle^+$
$\langle \text{FIGURE} \rangle$	$\rightarrow$	$\langle \text{GROUP} \rangle \mid \langle \text{POLY_2L} \rangle \mid \langle \text{BASE} \rangle$
$\langle \text{GROUP} \rangle$	$\rightarrow$	group $\langle \text{FIGURE} \rangle^2 \langle \text{FIGURE} \rangle^*$ end group
$\langle \text{POLY_2L} \rangle$	$\rightarrow$	poly2level $\langle \text{S_BASE} \rangle^3 \langle \text{S_BASE} \rangle^*$ end poly2level
$\langle \text{BASE} \rangle$	$\rightarrow$	$\langle \text{TRIANGLE} \rangle \mid \langle \text{POLYGON} \rangle$
$\langle \text{S_BASE} \rangle$	$\rightarrow$	$\langle \text{S_TRIANGLE} \rangle \mid \langle \text{S_POLYGON} \rangle$
$\langle \text{TRIANGLE} \rangle$	$\rightarrow$	triangle $\langle \text{POINT3} \rangle$ [$\langle \text{STYLE} \rangle$] end triangle
$\langle \text{POLYGON} \rangle$	$\rightarrow$	polygon $\langle \text{POINTS} \rangle$ [$\langle \text{STYLE} \rangle$] end polygon
$\langle \text{S_TRIANGLE} \rangle$	$\rightarrow$	triangle $\langle \text{POINT3} \rangle$ end triangle
$\langle \text{S_POLYGON} \rangle$	$\rightarrow$	polygon $\langle \text{POINTS} \rangle$ end polygon
$\langle \text{POINT3} \rangle$	$\rightarrow$	points = “ $n,n / n,n / n,n$ ”
$\langle \text{POINTS} \rangle$	$\rightarrow$	points = “ $n,n / n,n / n,n (/ n,n)^+$ ”
$\langle \text{STYLE} \rangle$	$\rightarrow$	style = $\langle \text{ASPECT_LST} \rangle$
$\langle \text{ASPECT_LST} \rangle$	$\rightarrow$	$\langle \text{FILL} \rangle ; \langle \text{STROKE} \rangle ; \langle \text{WIDTH} \rangle$ $\mid \langle \text{FILL} \rangle ; \langle \text{STROKE} \rangle$ $\mid \langle \text{FILL} \rangle ; \langle \text{WIDTH} \rangle$ $\mid \langle \text{STROKE} \rangle ; \langle \text{WIDTH} \rangle$ $\mid \langle \text{FILL} \rangle \mid \langle \text{STROKE} \rangle \mid \langle \text{WIDTH} \rangle$
$\langle \text{FILL} \rangle$	$\rightarrow$	fill : $\langle \text{COLOR} \rangle$
$\langle \text{STROKE} \rangle$	$\rightarrow$	stroke : $\langle \text{COLOR} \rangle$
$\langle \text{WIDTH} \rangle$	$\rightarrow$	width : n
$\langle \text{COLOR} \rangle$	$\rightarrow$	black $\mid$ red $\mid$ green $\mid$ none

The metasymbols are the same as in the grammar G_1 . Grammar G_2 is extended and not ambiguous, as it consists of non-ambiguous components. It would suffice to list the modified rules and the new ones.

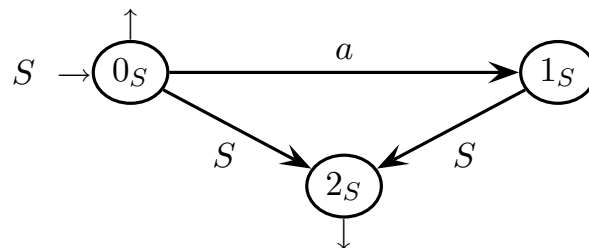
3 Analisi sintattica e parsificatori 20%

1. Si consideri la grammatica estesa (EBNF) G seguente, rappresentata come rete di macchine ricorsive con alfabeto $\{ a, b, S, X, Y \}$:



Si risponda alle domande seguenti:

- (a) Si tracci il grafo pilota $ELR(1)$ completo della grammatica G e se ne elenchino tutti gli eventuali conflitti.
- (b) Si analizzi la stringa $ababab \in L(G)$ tramite l'algoritmo di Earley applicato alla grammatica G , e se ne disegni l'albero sintattico. Si usi il quadro predisposto.
- (c) (facoltativa) Si consideri la grammatica estesa (EBNF) G' seguente, rappresentata come una sola macchina ricorsiva con alfabeto $\{ a, S \}$:



La grammatica G' è ambigua: si trovi una stringa ambigua generata da G' con (almeno) due derivazioni. La grammatica G' ha un conflitto di tipo $ELR(1)$: si spieghi di quale conflitto si tratta.

ATTENZIONE: si veda l'avviso sul frontespizio del testo.

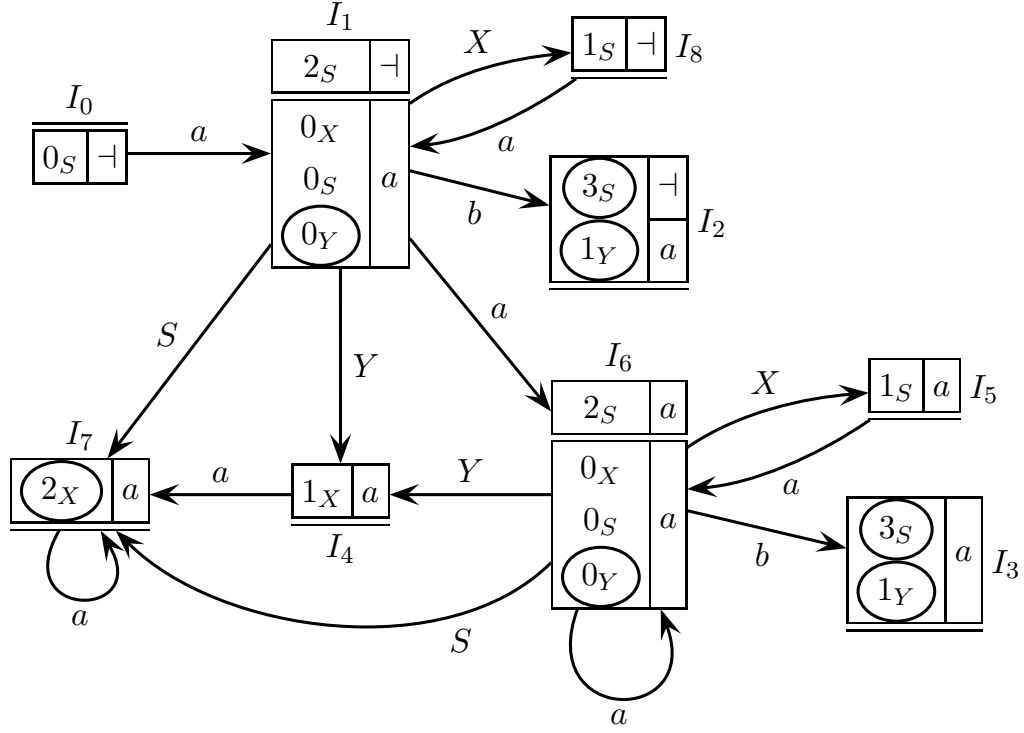
vettore di Earley per la stringa $a_1 b_2 a_3 a_4 b_5$

[illegible]

albero sintattico della stringa $a b a a b$

Soluzione

(a) Here is the pilot graph of grammar G , with nine m-states:



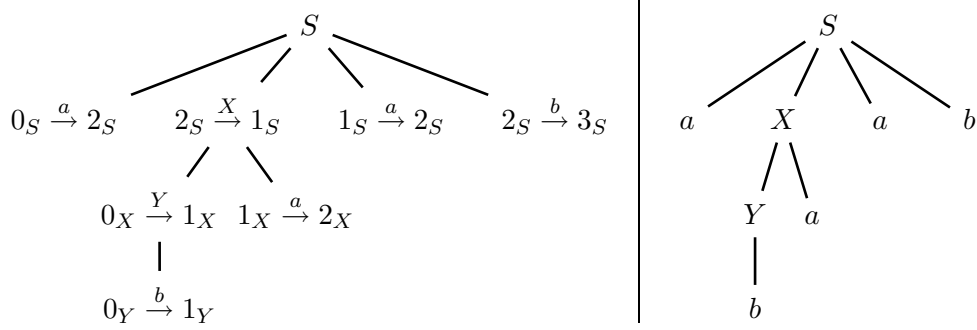
The pilot has one shift-reduce conflict in the m-states I_1 , I_6 and I_7 , and one reduce-reduce conflict in the m-state I_3 . Notice that the m-state I_2 contains two reduction candidates that do not cause a reduce-reduce conflict. Overall anyway, grammar G is not of type $ELR(1)$.

(b) Here is the Earley vector for the proposed string $ababab \in L(G)$:

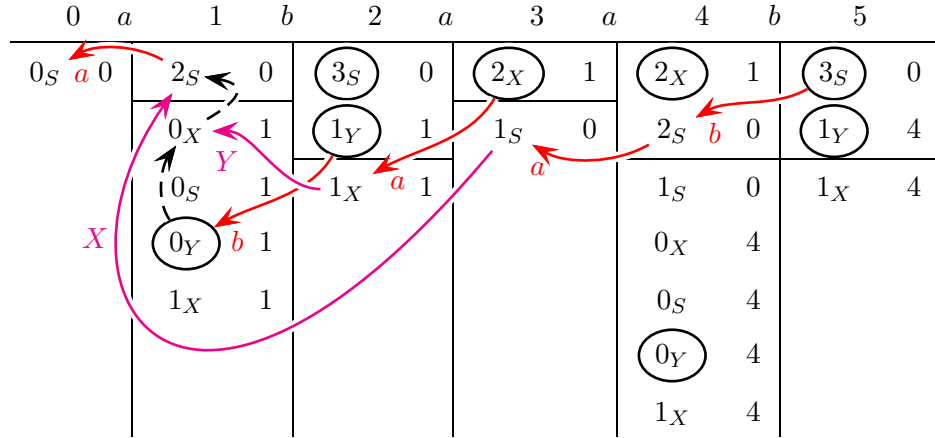
	0	a	1	b	2	a	3	a	4	b	5
0_S	0	2_S	0	(3_S)	0	(2_X)	1	(2_X)	1	(3_S)	0
		0_X	1	(1_Y)	1	1_S	0	2_S	0	(1_Y)	4
		0_S	1	1_X	1			1_S	0	1_X	4
		(0_Y)	1					0_X	4		
		1_X	1					0_S	4		
								(0_Y)	4		
								1_X	4		

The final candidate $\langle 3_S 0 \rangle$ in the last element $E[5]$ means acceptance, as expected. Notice that also the prefix string ab is accepted in the element $E[2]$.

The syntax tree is as follows (right):



We can see the tree backward reconstruction on the Earley vector, as follows:



The shift moves are coloured: red for terminal and magenta for nonterminal; and each arc label specifies the symbol (terminal or non) that is shifted. The black dashed arcs indicate the (backward) completion steps (left tree above).

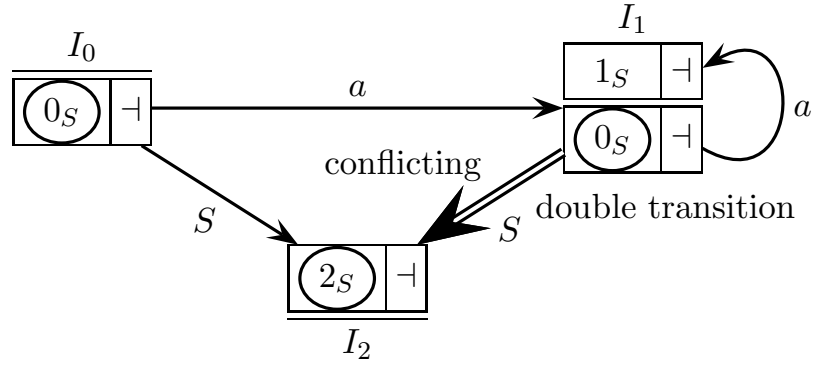
- (c) The machine graph of grammar G' is acyclic. It has three paths from the initial node 0_S , which correspond to the three alternative rules below:

$$G': S \rightarrow \varepsilon \quad S \rightarrow S \quad S \rightarrow a S$$

Grammar G' contains the identity rule $S \rightarrow S$, which of course could be simply canceled without changing the language. However such a rule is in the grammar and makes it ambiguous: every string of the language $L(G')$ has infinitely many derivations. For instance consider the empty string $\varepsilon \in L(G')$:

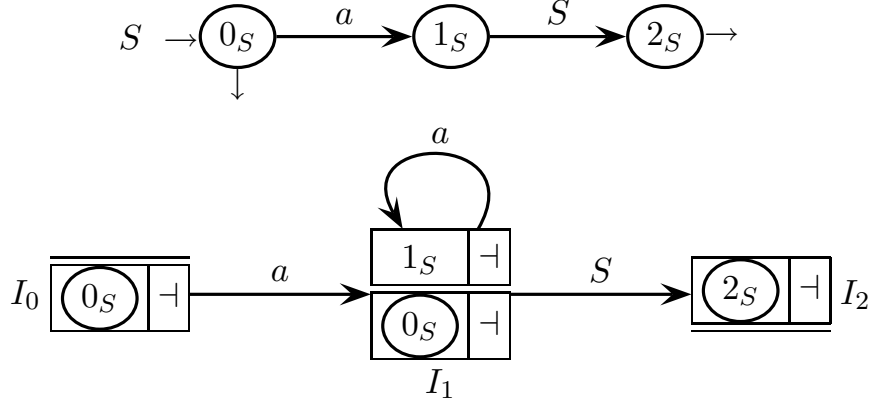
$$S \Rightarrow \varepsilon \quad S \Rightarrow S \Rightarrow \varepsilon \quad \text{and so on } \dots$$

Since grammar G' is (heavily) ambiguous, it has to have a conflict of type $ELR(1)$ (at least one if not two or more). A simple way to find it, is to try with the pilot graph, which is quite small and fast to build. Here it is:



The pilot does not have any shift-reduce or reduce-reduce conflicts. However the m-state transition $I_1 \xRightarrow{S} I_2$ is multiple, in particular double as it consists of the two candidate transitions $\langle 1_S \perp \rangle \xrightarrow{S} \langle 2_S \perp \rangle$ and $\langle 0_S \perp \rangle \xrightarrow{S} \langle 2_S \perp \rangle$. Both such transitions have the same destination state 2_S , so the m-state transition is convergent. But they have the same look-ahead $\perp$, so it is also conflicting. Therefore grammar G' has a convergence conflict, which makes G' non- $ELR(1)$. There are not any other conflicts, but this one suffices to break determinism.

A final observation: the language $L(G')$ is simply a^* (the universal language) and so is regular; therefore it can be generated by the right-linear grammar $G'': S \rightarrow \varepsilon \mid aS$, which is non-ambiguous and of type $ELR(1)$. Thus we see that grammar G'' is still grammar G' with the identity rule $S \rightarrow S$ canceled or, equivalently, the machine of G' with the arc $0_S \xrightarrow{S} 2_S$ stripped off. In fact the machine and the pilot graph of grammar G'' are these two:



It is immediate to see that the double transition in the pilot has turned into a single one, and therefore that the convergence conflict has disappeared. Thus the pilot does not have any conflict left, as expected.

4 Traduzione e analisi semantica 20%

1. Si consideri la traduzione $\tau: a^* \rightarrow q^* r^*$ seguente:

$$\tau(a^n) = q^{(n \div 3)} r^{(n \bmod 3)} \quad n \geq 0$$

dove gli operatori “ $\div$ ” e “ $\bmod$ ” calcolano quoziente e resto della divisione intera, rispettivamente. Per esempio:

$$\begin{aligned} \tau(\varepsilon) &= \varepsilon \\ \tau(a) &= r \\ \tau(aa) &= \tau(a^2) = r^2 = rr \\ \tau(aaa) &= \tau(a^3) = q \\ \tau(aaaa) &= \tau(a^4) = qr \\ \tau(aaaaa) &= \tau(a^5) = qr^2 = qrr \\ \tau(aaaaaa) &= \tau(a^6) = q^2 = qq \end{aligned}$$

Si risponda alle domande seguenti:

- (a) Si trovi un traduttore sequenziale deterministico T che calcoli la traduzione τ descritta sopra.
 (b) Si trovi un automa finito A a due ingressi che riconosca le coppie di stringhe (x, y) , dove si ha:

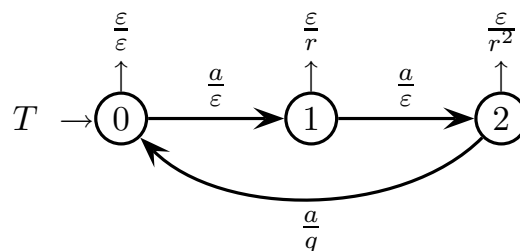
$$x = a^n \quad y = (\tau(x))^R = r^{(n \bmod 3)} q^{(n \div 3)} \quad n \geq 0$$

ossia la stringa y è l'immagine speculare della traduzione τ di x . Un esempio è la coppia valida $(aaaaa, rrrq) = (a^5, r^2q)$.

- (c) (facoltativa) Si scriva una grammatica di traduzione G_τ equivalente al traduttore T costruito al punto (a).

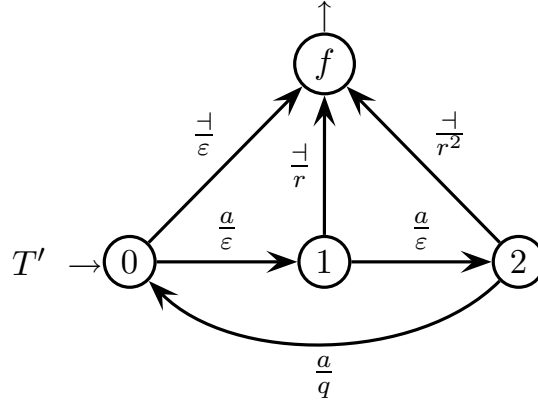
Soluzione

- (a) Here is a deterministic sequential transducer T' with an output function on the final nodes (actually here all the nodes are final):



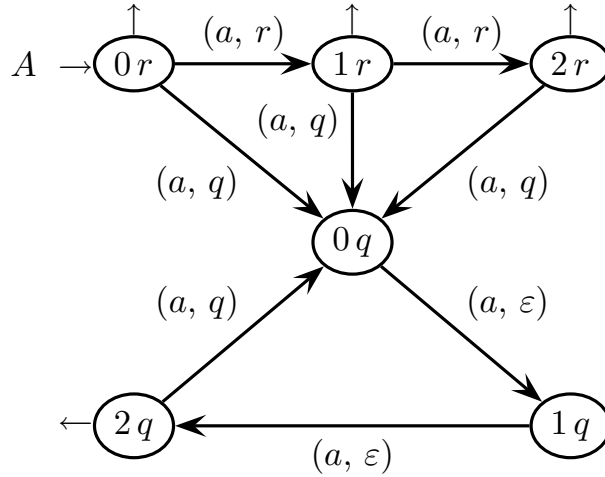
This transducer T is clearly deterministic because the subjacent recognizer is so.

Observation: there is a possible deterministic finite transducer T' without output function, but with a terminator $\dashv$. Here it is:



This transducer T' is also clearly deterministic because the subjacent recognizer is so. However the sequential transducer T above is smaller than T' .

- (b) Here is a possible finite automaton A with two input tapes:



As a two-input machine, this automaton A is deterministic, too, since there are not any two arcs from a state that have identical labels (remember that two couples are identical if their respective left and right members are equal).

- (c) Since translation τ can be computed by a finite transducer (see question (a)), it is regular (rational) and therefore there is a regular translation expression R_τ equivalent to τ . Here it is:

$$R_\tau = \left(\frac{a \ a \ a}{q} \right)^* \left(\frac{\varepsilon}{\varepsilon} \mid \frac{a}{r} \mid \frac{a \ a}{r \ r} \right)$$

Then it is immediate to transform expression R_τ into an equivalent translation grammar (or a scheme) G_τ of some kind. Here is a list of obvious possibilities:

$$\text{extended translation grammar} \quad \left\{ \begin{array}{l} S \rightarrow Q^* R \\ Q \rightarrow \frac{a \ a \ a}{q} \\ R \rightarrow \frac{\varepsilon}{\varepsilon} \mid \frac{a}{r} \mid \frac{a \ a}{r \ r} \end{array} \right.$$

$$\begin{array}{ll}
\text{non-extended translation grammar} & \left\{ \begin{array}{l} S \rightarrow Q R \\ Q \rightarrow \frac{a a a}{q} Q \mid \frac{\varepsilon}{\varepsilon} \\ R \rightarrow \frac{\varepsilon}{\varepsilon} \mid \frac{a}{r} \mid \frac{a a}{r r} \end{array} \right. \\
\text{right-linear translation grammar} & S \rightarrow \frac{a a a}{q} S \mid \frac{\varepsilon}{\varepsilon} \mid \frac{a}{r} \mid \frac{a a}{r r}
\end{array}$$

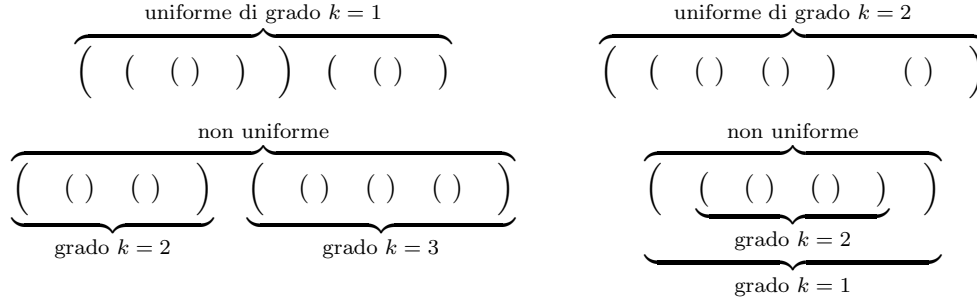
And possibly many others. These three are non-ambiguous.

A final observation: translation τ can be deterministically *computed* by a sequential transducer with input and output tapes, which emits the translated string onto the output tape (see question (a)); the reflected translation τ^R can be deterministically *verified* by a recognizer with two input tapes, which has the source string on the first input tape and the mirror image of the (supposed) string translated by τ on the second input tape, and which simply checks if this image corresponds to the source string (see question (b)); so there remains the question whether the reflected translation τ^R can be deterministically *computed* by a sequential transducer as well. The answer to the last question is negative: intuitively a deterministic sequential transducer has to scan the entire input tape for computing the division remainder, but then it will be too late for computing and emitting the quotient. The reader may check by himself that a nondeterministic sequential transducer is possible anyway: it suffices to convert the two-input recognizer into a sequential transducer - with the second input tape becoming the output tape - but at the price of making it nondeterministic.

2. Si considerino le strutture parentetiche annidate generate dalla sintassi seguente:

$$A \rightarrow A B \mid B \quad B \rightarrow '(' A ')' \mid '(' ' ')'$$

Una struttura parentetica è detta *uniforme di grado* $k \geq 1$ se ogni coppia non vuota di parentesi contiene immediatamente ed esattamente k coppie di parentesi. I quattro esempi qui sotto mostrano due strutture uniformi di grado 1 e 2, e due non uniformi.



Si aggiunga la regola assiomatica seguente alla sintassi (assioma S):

$$S \rightarrow A k$$

dove il simbolo terminale k specifica un grado ≥ 1 . Si può identificare tale terminale con un attributo che ha lo stesso nome k .

Si risponda alle domande seguenti:

- (a) Si scriva una grammatica con attributi per verificare se una struttura parentetica è uniforme di grado k . Si usino gli attributi seguenti:

<i>tipo</i>	<i>nome</i>	<i>associazione</i>	<i>dominio</i>	<i>significato</i>
destro	k	k	intero	grado richiesto da verificare sulla struttura
destro	d	$A B$	intero	grado da propagare per verificarlo sulla struttura
sinistro	u	$A B S$	booleano	$u = true$ se la struttura è uniforme di grado k , altrimenti $u = false$

Si possono aggiungere nuovi attributi, se è utile o necessario (anche per la domanda (b)). Si scrivano attributi e regole semantiche nei quadri predisposti.

- (b) (facoltativa) Si vuole calcolare un nuovo attributo *error* che dia la posizione della prima parentesi chiusa dove la condizione di uniformità è violata. L'attributo *error* è sinistro e visibile nella radice S ; in assenza di errori si ha $error = 0$. Così nell'esempio qui sotto, ponendo $k = 2$ tale parentesi ha la posizione $error = 8$:

$$\left(\begin{matrix} 1 \\ 2 \end{matrix} \left(\begin{matrix} 3 \\ 4 \end{matrix} \right) \left(\begin{matrix} 5 \\ 6 \end{matrix} \right) \right)_7 \right)_8$$

poiché la struttura 2-7 ha grado $2 = k$, ma la 1-8 ha grado $1 \neq k$ (3-4 e 5-6 sono vuote e non contano). Si scrivano le regole di calcolo, nel quadro predisposto.

Quadro degli attributi predefiniti e degli eventuali attributi da aggiungere o da completare per le domande (a) e (b) (il numero di righe vuote non è significativo):

<i>tipo</i>	<i>nome</i>	<i>associazione</i>	<i>dominio</i>	<i>significato</i>
destro	k	k	intero	grado richiesto da verificare sulla struttura
destro	d	$A \ B$	intero	grado da propagare per verificarlo sulla struttura
sinistro	u	$A \ B \ S$	booleano	$u = true$ se la struttura è uniforme di grado k , altrimenti $u = false$
sinistro	<i>error</i>	$A \ B \ S$	intero	posizione (≥ 1) della prima (ossia più a sinistra) parentesi chiusa dove la condizione di uniformità è violata; in assenza di errore l'attributo vale 0

#	sintassi	semantica - domanda (a) - da completare
		$d_1 := k_2$ $u_0 := u_1$
1:	$S_0 \rightarrow A_1 k_2$	
2:	$A_0 \rightarrow A_1 B_2$	
3:	$A_0 \rightarrow B_1$	
4:	$B_0 \rightarrow (A_1)$	
		$u_0 := true$
5:	$B_0 \rightarrow ()$	

#	<i>sintassi</i>	<i>semantica</i> - domanda (b) - da completare
		$d_1 := k_2$ $u_0 := u_1$ $error_0 := error_1$
1:	$S_0 \rightarrow A_1 k_2$	
2:	$A_0 \rightarrow A_1 B_2$	
3:	$A_0 \rightarrow B_1$	
4:	$B_0 \rightarrow (A_1)$	
		$u_0 := true$ $error_0 := 0$
5:	$B_0 \rightarrow ()$	

Soluzione

(a) Here are the attributes used (predefined / added):

<i>type</i>	<i>name</i>	<i>association</i>	<i>domain</i>	<i>meaning</i>
right	k	k	integer	degree required for verification on the structure
right	d	$A \ B$	integer	degree propagated for verification on the structure
left	u	$A \ B \ S$	boolean	$u = true$ if the structure is uniform with degree k , otherwise $u = false$
left	$error$	$A \ B \ S$	integer	position (≥ 1) of the first (i.e., leftmost) closed bracket where the violation of the uniformity condition gets apparent; in the absence of errors it has value 0
left	m	A	integer	computed local number of immediate bracket pairs
left	l	$A \ B$	integer	number of brackets encountered (string length)

Attribute k (formally right) is the value of terminal k , i.e., the required degree to be verified. Attribute d is inherited, initially takes the value of k and then propagates it downwards in order to compare it with the local degree of each parenthesis structure. Attribute m is synthesized and expresses the local number of immediate bracket pairs to be compared with the required degree (attribute d). In practice it counts the number of B 's generated by each A , as a B generates one parenthesis structure (possibly with inner structures). Finally attribute u is synthesized, and represents the uniformity predicate that is updated at each structure encountered and that is propagated upwards to the root.

And here are the semantic functions:

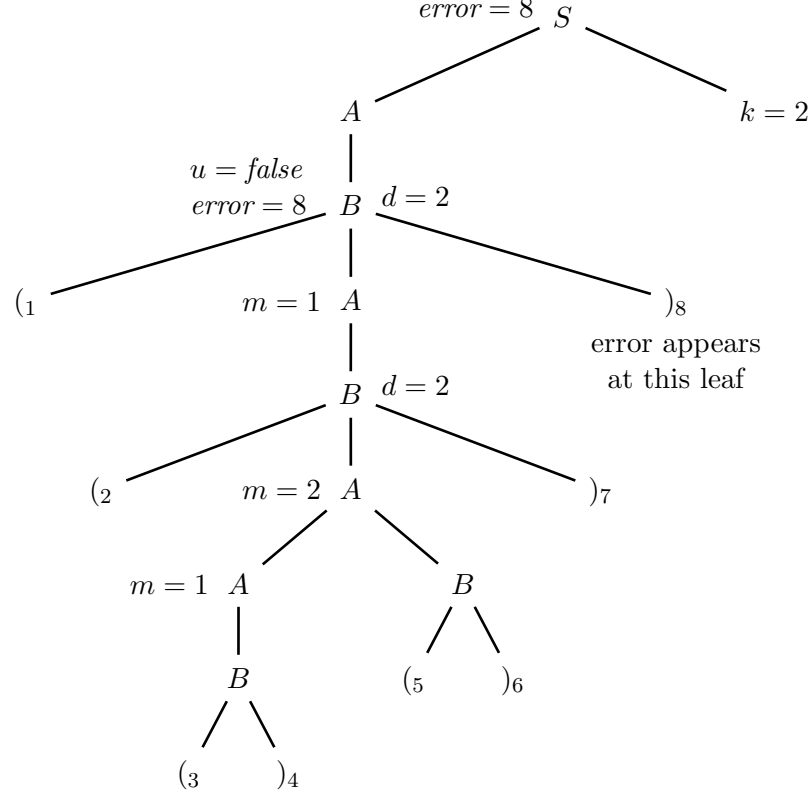
#	<i>syntax</i>	<i>semantic</i> - question (a) - completed
1:	$S_0 \rightarrow A_1 k_2$	$d_1 := k_2$ $u_0 := u_1$
2:	$A_0 \rightarrow A_1 B_2$	$d_1 := d_0$ $d_2 := d_0$ $m_0 := m_1 + 1$ $u_0 := u_1$ and u_2
3:	$A_0 \rightarrow B_1$	$d_1 := d_0$ $m_0 := 1$ $u_0 := u_1$
4:	$B_0 \rightarrow (A_1)$	$d_1 := d_0$ $u_0 := u_1$ and $(m_1 = d_0)$
5:	$B_0 \rightarrow ()$	$u_0 := true$

In the rule 4: $B_0 \rightarrow (A_1)$, the semantic assignment to attribute u_0 logically joins the attribute u_1 that expresses the status of the inner parenthesis structure generated by A_1 , and the equality comparison $m_1 = d_0$ that verifies the uniformity condition for structure (A_1) . If the status of the inner structure is erroneous, then attribute u_0 is set to *false*. If equality fails, then attribute u_0 is set to *false*. In both cases the presence of an error is propagated upwards. Otherwise there is not any error in the whole structure generated by B_0 and attribute u_0 is set to *true*. We might replace this assignment with a short code piece, perhaps more clear than a compact logical expression (left):

<pre> if $u_1 = false$ then $u_0 := false$ else if $m_1 \neq d_0$ then $u_0 := false$ else $u_0 := true$ end if assignment for rule 4 </pre>	<pre> if $u_1 = false$ then $u_0 := false$ else if $u_2 = false$ then $u_0 := false$ else $u_0 := true$ end if assignment for rule 2 </pre>
---	--

In the rule number 2 the statuses of the concatenated parenthesis structures generated by A and B , are logically joined and propagated upwards as well. See also the equivalent code piece above (right), which parallels the former one. The other semantic functions are quite obvious, just notice the inherited attribute d .

- (b) To see where the violation occurs on the tree and how it can be located, consider the proposed non-uniform example, where with $k = 2$ the violation is apparent at bracket number 8. Thus at the node B that generates bracket $)_8$, we expect to have $error = 8$, and up to the root (only the relevant attributes are shown):



At the first nonterminal B found starting from the tree root, attribute $error$ should be assigned the error position. Such an occurrence of B is the local root of rule 4: $B_0 \rightarrow (A_1)$. In the answer to question (a), we see that in the rule number 4 the semantic functions make an equality comparison between the computed local number of immediate bracket pairs generated by A_1 , i.e., attribute m_1 , and the required degree propagated from the root to B_0 , i.e., attribute d_0 . If equality fails, then the error gets apparent on setting attribute u_0 to *false* (see the assignment to u_0). So these are the appropriate place and condition where attribute $error$ could be assigned the position of the error closed bracket, provided a leftmost error closed bracket has not been found yet.

Attribute $error$ is synthesized. Initially, at the tree leaves, it can be left unspecified or, even better, set to value 0, which is not a valid position. Then it should be propagated upwards to the root and possibly assigned the first (leftmost) error position. If it reaches the root with a value still 0, then there is not any error, which is also apparent by assigning $u := true$; else there is, i.e., $u := false$. It is necessary to have also a synthesized integer attribute l for computing the position of the closed brackets (or computing the string length), which is much like counting the total number of bracket pairs encountered from left to right.

Therefore attribute *error* (and its companion *l*) is computed as follows:

#	<i>syntax</i>	<i>semantic</i> - question (b) - completed
1:	$S_0 \rightarrow A_1 k_2$	$d_1 := k_2$ $u_0 := u_1$ $error_0 := error_1$
2:	$A_0 \rightarrow A_1 B_2$	$d_1 := d_0$ $d_2 := d_0$ $m_0 := m_1 + 1$ $u_0 := u_1$ and u_2 if $error_1 \neq 0$ then $error_0 := error_1$ // left error else if $error_2 \neq 0$ then $error_0 := l_1 + error_2$ // right error else $error_0 := 0$ // no error end if $l_0 := l_1 + l_2$
3:	$A_0 \rightarrow B_1$	$d_1 := d_0$ $m_0 := 1$ $u_0 := u_1$ $error_0 := error_1$ $l_0 := l_1$
4:	$B_0 \rightarrow (A_1)$	$d_1 := d_0$ $u_0 := u_1$ and $(m_1 = d_0)$ if $error_1 \neq 0$ then $error_0 := error_1$ // inner error else if $m_1 \neq d_0$ then $error_0 := l_1 + 2$ // outer error else $error_0 := 0$ // no error end if $l_0 := l_1 + 2$
5:	$B_0 \rightarrow ()$	$u_0 := true$ $error_0 := 0$ $l_0 := 2$

In the rule number 2 the error closed bracket that may occur in the left parenthesis structure generated by A_1 , prevails on the error closed bracket that may occur in the right structure generated by B_2 . In fact we want to signal the first occurrence of uniformity violation, i.e., the leftmost one.

Similarly in the rule number 4, the error closed bracket that may occur in the inner parenthesis structure generated by A_1 , prevails on the error closed bracket

that is locally generated by B_0 . In fact the inner structure has a closed bracket on the left of that of the outer one.

Attribute l is synthesized and its computation is straightforward: it is the string length and can be calculated by pairwise counting the brackets. The other semantic functions are the same as in the question (a).

Notice that this solution is a generalization of that of question (a), with a few semantic functions added (in red), which are independent of attribute u . Notice also that attribute *error* semantically contains attribute u , as we have the logical equivalence $(error = 0) \Leftrightarrow (u = true)$ at all the nodes. In fact the similarity of the code pieces for computing attributes u and *error*, is striking. Actually attribute u simply detects an error, while attribute *error* locates it. Consequently we might suppress attribute u . However we might keep both attributes u and *error*, but we could partially merge their computations since they share some conditions.

A final observation (not requested by the exercise): both attribute grammars are one-sweep. In fact most attributes are synthesized and the only two inherited ones, which are k and d , depend on a terminal or only on inherited attributes of the parent node. The reader may formally check by himself.